Hierarchical Modularity & Contracts

This chapter develops the modular view of digital systems as collections of components with explicit interfaces, together with the contracts that govern their composition. The guiding idea is that a module is not merely a piece of implementation, but a reusable artifact whose behavior is constrained at its boundary. When modules are assembled hierarchically, the resulting system should preserve the invariants stated at each level of abstraction, and those invariants should be checkable by construction.

A useful starting point is the boundary error metric $\varepsilon(r)$, interpreted as the probability that a contract is violated at an interface or other composition boundary. Here r may be read as a resolution parameter, a refinement level, or a resource scale, depending on the surrounding model. The point of introducing $\varepsilon(r)$ is not to replace logical correctness with a probabilistic surrogate, but to make explicit how often a design departs from its declared specification when it is exercised at a given scale. In a hierarchical setting, one expects such boundary error to decrease under refinement, or at least to be controlled by a compositional bound.

Modules are characterized by their interfaces, parameterization, and reusability conditions. An interface specifies the data, signals, or morphisms that may cross the module boundary, while parameterization records the family of behaviors obtained by varying admissible inputs or configuration values. Reusability requires more than syntactic compatibility: a module should be substitutable in a larger assembly without invalidating the assumptions made by its environment. In contract-based design, this is expressed by a pair of obligations. The module assumes certain properties of its context and guarantees certain properties in return. Composition is valid when the guarantees of one component satisfy the assumptions of the next.

This perspective aligns naturally with categorical descriptions of open systems. In particular, hypergraph categories and decorated cospans provide a formal language for open circuits and other boundary-bearing structures. A decorated cospan represents a system together with designated input and output interfaces, while the decoration records the internal structure or semantics of the open component. Hypergraph categories supply the algebraic operations needed to compose such systems in parallel and in series, while retaining explicit boundary data. In this setting, contracts can be attached to interfaces and propagated through composition, so that the semantics of a larger circuit is assembled from the semantics of its parts.

The central technical requirement is that refinement maps preserve the intended invariants across levels of abstraction. A refinement map relates a coarse specification to a more detailed implementation, or one module version to a more concrete realization. If the map is well chosen, then properties proved at the abstract level remain valid after refinement, possibly after translation into the language of the lower level. This is the mechanism by which hierarchical invariants compose: one proves a property once at a high level, then shows that each refinement step respects the property, so that the final implementation inherits it.

In practice, hierarchical invariants are most useful when they are stable under the operations used to build systems. If two modules each satisfy their local contracts, then their composite should satisfy a derived contract. Likewise, if a module is refined into a more detailed implementation, the refined module should satisfy the original contract or a strengthening of it. These closure properties are the modular analogue of algebraic closure: the class of admissible components remains closed under the composition rules of the design language.

The chapter therefore treats contracts not as informal documentation, but as executable specifications that can be checked in a build pipeline. A contract-checked library is an artifact in which module interfaces, refinement relations, and invariant checks are encoded so that continuous integration can validate them automatically. Such a library serves two purposes. First, it demonstrates

that the abstract theory can be instantiated in a concrete engineering workflow. Second, it provides a reproducible testbed for studying how boundary errors, refinement maps, and compositional invariants behave under change.

The intended artifact for this chapter is a contract-checked library with CI tests. The library should expose a small collection of modules with explicit interfaces, a refinement relation between at least two abstraction levels, and automated tests that verify the stated contracts at composition boundaries. The tests should report whether the observed boundary behavior is consistent with the chosen $\varepsilon(r)$ criterion, and they should fail when a contract is violated. In this way, the chapter connects the categorical language of open systems with the practical discipline of buildable, checkable modular software and hardware design.

Taken together, these ideas establish the chapter's thesis: hierarchical modularity is strongest when interfaces are explicit, refinement is structure-preserving, and contracts are enforced by executable checks. Under those conditions, compositional invariants are not merely asserted; they are maintained across levels of abstraction and across the assembly of open components.

To make the discussion operational, it is useful to distinguish three layers of specification. At the outer layer, an interface declares the observable ports of a module and the admissible directions of information flow. At the middle layer, a contract states the assumptions on the environment and the guarantees offered by the module. At the inner layer, an implementation realizes the contract by concrete code, circuit structure, or state transition rules. A refinement map connects these layers by translating abstract obligations into lower-level conditions that can be checked mechanically. When the translation is sound, the implementation is not merely similar to the specification; it is a witness that the specification has been realized.

This layered view also clarifies reusability conditions. A module is reusable when its interface is stable under substitution, its contract is sufficiently local to be checked at the boundary, and its parameterization does not leak hidden dependencies into the surrounding system. In categorical terms, the module should behave functorially with respect to the chosen notion of context: replacing one compatible component by another should preserve the composite's well-formedness. In engineering terms, the same principle appears as encapsulation, dependency inversion, and interface-driven development.

The role of $\varepsilon(r)$ is especially important when exact boundary satisfaction is too strict to be informative at every scale. For example, a coarse model may admit a small but nonzero probability of mismatch at an interface, while a refined model reduces that mismatch by tightening the implementation or by increasing the resolution of the test harness. The metric therefore acts as a bridge between abstract contract satisfaction and empirical validation. It records how far the observed behavior departs from the declared boundary condition, and it provides a quantitative target for refinement.

Hypergraph categories and decorated cospans supply a particularly apt language for this bridge because they separate boundary structure from internal decoration. The boundary is what contracts talk about; the decoration is what refinement changes. Composition glues boundaries while combining decorations, so the formalism naturally supports modular reasoning. If each decorated component satisfies its local contract, then the composite can be assigned a derived contract whose validity follows from the compositional laws of the category. This is the mathematical expression of the engineering principle that local correctness should scale to system correctness.

The chapter's artifact requirement is therefore not incidental but constitutive of the exposition. A contract-checked library with CI tests makes the abstract claims inspectable. It should include at least one example of a coarse module and a refined module, a specification of the refinement map between them, and a test suite that exercises the interfaces under composition. The continuous integration workflow should record whether the contract checks pass, whether any boundary

violations are observed, and how those observations compare with the chosen $\varepsilon(r)$ threshold. Such a repository becomes a living proof object: the theory is encoded in the design, and the design is validated by the theory.

In summary, hierarchical modularity is the discipline of organizing systems so that interfaces, contracts, and refinement maps cooperate. The categorical viewpoint explains why this organization is compositional; the contract viewpoint explains how it is checkable; and the artifact viewpoint explains how it is made reproducible. The chapter develops these themes as a foundation for the later treatment of sequential composition, feedback, and traced structure, where the same emphasis on boundaries and invariants will reappear in a more dynamic setting.